Design and Implementation of a Bipedal Walker Using Hybrid Zero Dynamics and Optimization-based Gait

EECE 7398: Project Report

Theo McArn
mcarn.t@northeastern.edu

Arjun Viswanathan
viswanathan.ar@northeastern.edu

RM Shahriar Hoque
hoque.r@northeastern.edu

Tong Sha
sha.t@northeastern.edu

April 19, 2026

Abstract

We present a complete design and simulation pipeline for a three-link planar bipedal robot using Hybrid Zero Dynamics (HZD). The robot is modeled as an underactuated mechanical system with a cyclic stance leg angle and two actuated joints at the hip. We derive the full Lagrangian equations of motion, design virtual constraints parameterized by Bezier polynomials, and reduce the closed-loop dynamics to a two-dimensional zero dynamics system on a constraint manifold. A gradient-based optimizer finds Bezier coefficients and pre-impact conditions that minimize actuator effort while enforcing a periodic orbit. We then enforce these virtual constraints using a closed-loop feedback linearization controller. The final result is stable control of an underactuated bipedal robot over a 10 step trajectory.

1 Introduction

Bipedal locomotion is one of the harder problems in robotics. Unlike wheeled systems, legged robots have to actively manage their own balance at every step, and unlike fully actuated manipulators, they deal with intermittent ground contact, underactuation during the swing phase, and impulsive forces at footstrike. All of these factors make designing a stable walking gait a non-trivial, and highly nonlinear problem.

The Hybrid Zero Dynamics (HZD) framework, developed by Grizzle et. al. [1], is one of the cleanest ways to handle this. The core idea is to use feedback control to enforce geometric relationships between joint angles, called *virtual constraints*, so that the full system's behavior is determined by a much lower-dimensional set of equations called the *zero dynamics*. Once you have a 2D system instead of a 6D one, you can run an optimizer on it efficiently and find gaits that are periodic, stable, and energy-efficient.

In this project we implement HZD from the ground up for a three-link planar biped. We derive the dynamics symbolically, implement a feedback linearization controller, reduce the swing-phase dynamics to zero dynamics, and use the `fmincon` optimizer in MATLAB to find an optimal periodic gait. The whole pipeline is validated by simulating the full closed-loop system over multiple steps.

2 System Model

2.1 Mechanical Configuration and Generalized Coordinates

The robot is modeled as three rigid links in the sagittal plane: a stance leg of length r and point mass m_1, a swing leg of length r and point mass m_2, and a torso of length l and point mass M_t, with a point mass M_h at the hip (Figure 1). The stance leg is pinned to the ground at its foot during each swing phase.

We use three generalized coordinates:

- q_1: absolute angle of the stance leg measured from the upward vertical (positive y-axis)
- q_2: angle of the swing leg relative to q_1
- q_3: angle of the torso relative to q_1

By convention, $q_1 > 0$ when the stance leg leans backward (just after toe-off) and $q_1 < 0$ at impact. The complete state vector is $x = (q, \dot{q}) \in \mathbb{R}^6$ with $q = [q_1, q_2, q_3]^\top$ and $\dot{q} = [\dot{q}_1, \dot{q}_2, \dot{q}_3]^\top$.

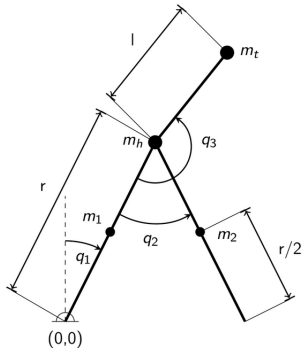

Figure 1: Three-link planar bipedal walker. The stance foot is fixed during the swing phase. Angles q_1, q_2, q_3 are the generalized coordinates.

The numerical parameters used throughout this work are summarized in Table 1.

Table 1: Model parameters

Parameter	Symbol	Value	Units
Leg length	r	1.0	m
Leg mass (each)	m	5.0	kg
Hip mass	M_h	15.0	kg
Torso mass	M_t	10.0	kg
Torso length	l	0.5	m
Gravity	g	9.81	m/s^2

2.2 Forward Kinematics

All positions are expressed in an inertial frame with origin at the stance foot. We use the convention that rotations are measured from the positive y-axis (upward vertical), so the SO(2) rotation matrix is:

$$R(\theta) = \begin{bmatrix} \sin\theta & -\cos\theta \\ \cos\theta & \sin\theta \end{bmatrix}. \tag{1}$$

The position vectors of the relevant mass centers are:

Hip (M_h, at the top of the stance leg):

$$p_{M_h} = r \begin{bmatrix} \sin(-q_1) \\ \cos(-q_1) \end{bmatrix} = r \begin{bmatrix} -\sin q_1 \\ \cos q_1 \end{bmatrix} \tag{2}$$

Torso center of mass (M_t, along the torso from the hip):

$$p_{M_t} = p_{M_h} + l \begin{bmatrix} \sin(-q_1 + \pi - q_3) \\ \cos(-q_1 + \pi - q_3) \end{bmatrix} \tag{3}$$

Stance leg midpoint (mass m, at $r/2$ along the stance leg):

$$p_{m_1} = \frac{r}{2} \begin{bmatrix} -\sin q_1 \\ \cos q_1 \end{bmatrix} \tag{4}$$

Swing leg midpoint (mass m, at $r/2$ from the hip along the swing leg):

$$p_{m_2} = p_{M_h} + \frac{r}{2} \begin{bmatrix} \sin(q_1 + q_2) \\ -\cos(q_1 + q_2) \end{bmatrix} \tag{5}$$

Swing foot (end of the swing leg, used for impact detection):

$$p_2 = p_{M_h} + r \begin{bmatrix} \sin(q_1 + q_2) \\ -\cos(q_1 + q_2) \end{bmatrix} \tag{6}$$

Velocities are obtained by differentiating the position vectors with respect to time:

$$v_i = \frac{\partial p_i}{\partial q} \dot{q} = J_i(q)\, \dot{q}, \tag{7}$$

where $J_i = \partial p_i / \partial q \in \mathbb{R}^{2\times 3}$ is the Jacobian of the i-th mass.

2.3 Equations of Motion: Swing Phase

During the swing phase, the system obeys the Euler-Lagrange equations of motion, derived from the Lagrangian $\mathcal{L} = K - V$:

$$\frac{d}{dt}\frac{\partial \mathcal{L}}{\partial \dot{q}} - \frac{\partial \mathcal{L}}{\partial q} = Bu, \tag{8}$$

which can be written in standard manipulator form as:

$$D(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) = Bu, \tag{9}$$

where:

- $D(q) \in \mathbb{R}^{3\times 3}$ is the symmetric positive-definite mass inertia matrix
- $C(q,\dot{q}) \in \mathbb{R}^{3\times 3}$ is the Coriolis/centrifugal matrix (computed via Christoffel symbols)
- $G(q) \in \mathbb{R}^3$ is the gravity vector ($\partial V / \partial q$)
- $B = [0\ 0;\ 1\ 0;\ 0\ 1]^\top \in \mathbb{R}^{3\times 2}$ is the input selection matrix (no actuation on q_1)
- $u \in \mathbb{R}^2$ is the control torque vector applied at joints q_2 and q_3

2.3.1 Mass Inertia Matrix

The total kinetic energy can be written as a quadratic form in $\dot{q}$:

$$K = \frac{1}{2}\dot{q}^\top D(q)\dot{q}, \tag{10}$$

where $D(q)$ is the symmetric, positive-definite mass inertia matrix. It then follows that we can solve for $D(q)$ by taking the Hessian of Kinetic Energy:

$$D(q) = \frac{\partial^2 K}{\partial \dot{q}^2}. \tag{11}$$

Calculating $D(q)$ for our specific configuration results in the following 3×3 matrix:

$$D_{11} = \frac{3mr^2}{2} + M_h r^2 + M_t r^2 + M_t l^2 - mr^2 \cos q_2 - 2M_t r l \cos q_3 \tag{12}$$

$$D_{12} = D_{21} = -\frac{mr^2(2\cos q_2 - 1)}{4} \tag{13}$$

$$D_{13} = D_{31} = M_t l(l - r\cos q_3) \tag{14}$$

$$D_{22} = \frac{mr^2}{4} \tag{15}$$

$$D_{23} = D_{32} = 0 \tag{16}$$

$$D_{33} = M_t l^2 \tag{17}$$

2.3.2 Coriolis Matrix

The Coriolis matrix is assembled from Christoffel symbols of the first kind:

$$C_{kj} = \sum_{i=1}^{3} \frac{1}{2}\left(\frac{\partial D_{kj}}{\partial q_i} + \frac{\partial D_{ki}}{\partial q_j} - \frac{\partial D_{ij}}{\partial q_k}\right)\dot{q}_i. \tag{18}$$

The nonzero entries are:

$$C_{11} = \frac{mr^2 \sin q_2}{2}\dot{q}_2 + M_t rl \sin q_3\, \dot{q}_3 \tag{19}$$

$$C_{12} = \frac{mr^2 \sin q_2}{2}(\dot{q}_1 + \dot{q}_2) \tag{20}$$

$$C_{13} = M_t rl \sin q_3\, (\dot{q}_1 + \dot{q}_3) \tag{21}$$

$$C_{21} = -\frac{mr^2 \sin q_2}{2}\dot{q}_1 \tag{22}$$

$$C_{31} = -M_t rl \sin q_3\, \dot{q}_1 \tag{23}$$

All other entries are zero.

2.3.3 Gravity Vector

The gravity vector is the gradient of potential energy with respect to configuration:

$$G = \frac{\partial V}{\partial q} \tag{24}$$

with:

$$G_1 = -\frac{g}{2}\big[2M_h r \sin q_1 + 2M_t r \sin q_1 + 3mr \sin q_1 - 2M_t l \sin(q_1 + q_3) - mr \sin(q_1 + q_2)\big] \tag{25}$$

$$G_2 = \frac{gmr \sin(q_1 + q_2)}{2} \tag{26}$$

$$G_3 = M_t gl \sin(q_1 + q_3) \tag{27}$$

These matrices are derived symbolically in `genrate_functions.m` using MATLAB's Symbolic Toolbox and written to `autogen/func_compute_D_C_G_B.m`.

2.4 Impact Map

When the swing foot touches the ground, completing the gait cycle (detected when the gait timing variable $s \to 1$), the system undergoes an instantaneous impact. We assume that the contact is perfectly inelastic with no rebound or slippage at the impacting foot, so the velocity of the new stance foot must be zero immediately after impact. While configurations are unchanged through impact, velocities are discontinuous.

To formulate the impact map, the minimal coordinates $q \in \mathbb{R}^3$ are insufficient: enforcing zero absolute velocity at the swing foot requires knowledge of the robot's translational motion, not just its joint angles. We therefore introduce an unpinned $(N{+}2)$-DOF model with extended coordinates $q_e = [q^\top, p_h, p_v]^\top \in \mathbb{R}^5$, where $p_e = (p_h, p_v)$ is a fixed reference point (e.g. the stance foot) in Cartesian space.

The extended inertia matrix $D_e \in \mathbb{R}^{5\times 5}$ is computed analogously to D but with respect to $\dot{q}_e$:

$$D_e(q_e) = \frac{\partial^2 K_e}{\partial \dot{q}_e^2}, \tag{28}$$

and the contact Jacobian $E_2 \in \mathbb{R}^{2\times 5}$ maps extended velocities to the swing foot velocity:

$$E_2(q_e) = \frac{\partial p_2^e}{\partial q_e}, \tag{29}$$

where p_2^e is the position of the swing foot tip in extended coordinates.

The zero-velocity constraint at impact is then

$$E_2(q_e^-)\dot{q}_e^+ = 0 \tag{30}$$

In addition to this constraint, momentum must also be conserved through the instantaneous impact. This provides a second constraint equation:

$$D_e(q_e^-)\dot{q}_e^+ - F_{\text{ext}} = D_e(q_e^-)\dot{q}_e^- \tag{31}$$

Combining these two constraints yields the block-matrix system:

$$\begin{bmatrix} D_e(q_e^-) & -E_2^\top(q_e^-) \\ E_2(q_e^-) & 0 \end{bmatrix} \begin{bmatrix} \dot{q}_e^+ \\ F_2 \end{bmatrix} = \begin{bmatrix} D_e(q_e^-)\dot{q}_e^- \\ 0 \end{bmatrix}, \tag{32}$$

where F_2 is the impulsive contact force at the new stance foot. Solving this system gives:

$$\Delta F_2 = -\left(E_2 D_e^{-1} E_2^\top\right)^{-1} E_2 \begin{bmatrix} I_3 \\ \frac{\partial}{\partial q_s}\Upsilon_e \end{bmatrix}, \tag{33}$$

$$\bar{\Delta}_{\dot{q}_e} = D_e^{-1} E_2^\top \Delta F_2 + \begin{bmatrix} I_3 \\ \frac{\partial}{\partial q_s}\Upsilon_e \end{bmatrix}, \tag{34}$$

where $\Upsilon_e(q_s)$ recovers p_e from the minimal coordinates via forward kinematics.

Lastly, since the swing foot becomes the new stance foot after impact, the relabeling matrix R is applied, which exchanges the roles of the stance and swing legs:

$$q^+ = R\, q^-, \qquad \dot{q}^+ = [R,\ \mathbf{0}_{3\times 2}]\, \bar{\Delta}_{\dot{q}_e}(q_s^-)\, \dot{q}_s^-, \tag{35}$$

$$R = \begin{bmatrix} 1 & 1 & 0 \\ 0 & -1 & 0 \\ 0 & -1 & 1 \end{bmatrix}, \qquad R^2 = I. \tag{36}$$

The involutory property $R^2 = I$ reflects the physical symmetry of leg exchange.

For the impact model to be physically valid, the following conditions must hold at each impact:

1. $F_2^N > 0$: the normal contact force is compressive (the ground pushes up),

2. $|F_2^T| \leq \mu_s F_2^N$: the tangential force lies within the friction cone (no slipping),

3. $\dot{p}_1^v \geq 0$: the old stance foot does not penetrate the ground post-impact.

If any of these conditions are violated, the gait must be redesigned.

The impact map is implemented in `func_impact_map.m` and the extended matrices are computed in `autogen/func_compute_De_E_dY_dq.m`.

2.5 Hybrid Dynamics

The complete walking model is a hybrid dynamical system that alternates between continuous swing-phase dynamics and discrete impact events:

$$\Sigma : \begin{cases} \dot{x} = f_s(x) + g_s(x)\,u, & x^- \notin \mathcal{S} \\ x^+ = \Delta(x^-), & x^- \in \mathcal{S} \end{cases} \tag{37}$$

where $x = (q, \dot{q}) \in T\mathcal{Q}$ is the state. The switching surface $\mathcal{S}$ detects the end of the swing phase:

$$\mathcal{S} = \left\{ (q, \dot{q}) \in T\mathcal{Q} \;\middle|\; p_2^v(q) = 0,\ p_2^h(q) > 0 \right\}, \tag{38}$$

where p_2^v and p_2^h are the vertical and horizontal positions of the swing foot, respectively. The two conditions ensure that impact is triggered only when the swing foot has reached the ground and is ahead of the stance foot. The impact map Δ applies the velocity update (35) and relabeling (36) to produce the post-impact state x^+, which serves as the initial condition for the next swing phase.

3 Virtual Constraints and Gait Design

Now that we have modeled our system with hybrid dynamics, we need to design a stable gait, parameterized by the gait timing variable which can produce a stable walking movement.

3.1 Gait Timing Variable

A key insight of HZD is that the desired joint trajectories should be functions of the robot's own configuration rather than time. This makes the controller *time-invariant* and robust to disturbances that change the walking speed. This gait timing variable, s, which has a range of [0,1] can be a function of any variable within the configuration. The only constraint is that it is monotonic. We use the stance leg angle q_1 as this cyclic variable through the normalized *gait timing variable*:

$$s(q_1) = \frac{q_1 - q_1^+}{q_1^- - q_1^+} \in [0, 1] \tag{39}$$

where q_1^+ is the post-impact value of q_1 (start of swing) and q_1^- is the pre-impact value (end of swing). As the robot walks forward, q_1 moves from q_1^+ to q_1^-, sweeping s from 0 to 1.

3.2 Bezier Parameterization

The desired trajectories for the actuated joints are expressed as 4th degree Bezier polynomials in s:

$$b_i(s) = \sum_{k=0}^{4} \alpha_k^i \binom{4}{k} s^k (1-s)^{4-k} \tag{40}$$

where α_k^i are the control points (Bezier coefficients) for joint $i \in \{2, 3\}$. Bezier polynomials are convenient here because their boundary values are exactly the first and last control points:

$$b_i(0) = \alpha_0^i, \qquad b_i(1) = \alpha_4^i \tag{41}$$

and the endpoint slopes are proportional to adjacent coefficient differences. This makes it easy to encode boundary conditions at the start and end of each step, which we will utilize when defining initial conditions for the optimizer.

Additionally, Bezier polynomials are easily differentiable, making it trivial to derive not only q but also $\dot{q}$ from $b(s)$.

The first two coefficients (α_0^i, α_1^i) are not free parameters, they are determined by periodicity conditions that ensure the Bezier curve is consistent across the impact transition. At the end of a step ($s = 1$), the pre-impact configuration is relabeled by R to produce the post-impact configuration of the next step ($s = 0$). Matching both position and velocity through this transition yields the constraints:

$$\alpha_0^i = -\alpha_4^i + 2\alpha_2^i, \qquad \alpha_1^i = -\alpha_3^i + 2\alpha_2^i \tag{42}$$

This leaves only $\{\alpha_2^i, \alpha_3^i, \alpha_4^i\}$ as free parameters for each joint, six numbers total across both joints.

3.3 Output Function and Virtual Constraints

The output function is:

$$h(x) = \begin{bmatrix} q_2 - b_2(s(q_1)) \\ q_3 - b_3(s(q_1)) \end{bmatrix} \tag{43}$$

The virtual constraints $h(x) = 0$ define a 4-dimensional constraint surface in the 6-dimensional state space. When the feedback controller enforces $h = 0$, the joint angles q_2 and q_3 are fully determined by q_1 through the Bezier curves. The robot then only has one degree of freedom left: q_1 itself.

4 Feedback Linearization

4.1 Control-Affine Form

The closed-loop system can be written in control-affine form $\dot{x} = f(x) + g(x)u$ where $x = [q^T,\, \dot{q}^T]^T \in \mathbb{R}^6$:

$$f(x) = \begin{bmatrix} \dot{q} \\ D^{-1}(-C\dot{q} - G) \end{bmatrix}, \quad g(x) = \begin{bmatrix} 0_{3\times 2} \\ D^{-1}B \end{bmatrix} \tag{44}$$

4.2 Lie Derivative Control Law

Since the output h has *relative degree two* (the input u first appears in $\ddot{h}$), feedback linearization requires two Lie derivatives. The first Lie derivative $L_f h = \dot{h}$ along f is:

$$L_f h = \frac{\partial h}{\partial x} f(x) \tag{45}$$

The key subtlety is that h depends on q_1 through $s(q_1)$, so the chain rule gives:

$$\frac{\partial h_i}{\partial q_1} = -\frac{1}{\Delta q}\frac{\partial b_i}{\partial s} \tag{46}$$

where $\Delta q = q_1^- - q_1^+$. The second Lie derivative $L_f^2 h$ and the mixed derivative $L_g L_f h$ (how the input enters $\ddot{h}$) are computed from the Jacobian of $L_f h$:

$$L_f^2 h = \frac{\partial (L_f h)}{\partial x} f(x), \quad L_g L_f h = \frac{\partial (L_f h)}{\partial x} g(x) \tag{47}$$

These are precomputed symbolically and stored as `func_compute_dLfh`.

With a virtual PD input $v = -K_p h - K_d L_f h$, the feedback linearizing control law is:

$$u = (L_g L_f h)^{-1}(v - L_f^2 h) \tag{48}$$

This drives $\ddot{h} = v$, so the output error decays exponentially with gains $K_p = 100$, $K_d = 20$ (critically damped at $\omega_n \approx 10$ rad/s). Once the transients die out, $h \approx 0$ and $\dot{h} \approx 0$.

5 Zero Dynamics

5.1 The Zero Dynamics Manifold

The *zero dynamics manifold* $\mathcal{Z}$ is the set of states where both the output and its derivative are zero:

$$\mathcal{Z} = \{x \in \mathbb{R}^6 \mid h(x) = 0,\ L_f h(x) = 0\} \tag{49}$$

On $\mathcal{Z}$, the virtual constraints hold exactly, so $q_2 = b_2(s)$ and $q_3 = b_3(s)$. This means all six states are determined by just two numbers: q_1 and $\dot{q}_1$. The full state space collapses to 2D.

5.2 Deriving the Zero Dynamics

On $\mathcal{Z}$, differentiating $q_2 = b_2(s(q_1))$ twice gives:

$$\ddot{q}_2 = \underbrace{\frac{\partial^2 b_2}{\partial s^2}\left(\frac{\dot{q}_1}{\Delta q}\right)^2}_{\beta_1^{(2)}} + \underbrace{\frac{\partial b_2}{\partial s}\frac{1}{\Delta q}}_{\beta_2^{(2)}} \ddot{q}_1 \tag{50}$$

and similarly for q_3. Collecting these into vector form, the body acceleration on $\mathcal{Z}$ is:

$$\begin{bmatrix} \ddot{q}_2 \\ \ddot{q}_3 \end{bmatrix} = \beta_1 + \beta_2\, \ddot{q}_1 \tag{51}$$

where $\beta_1 \in \mathbb{R}^2$ is the curvature-velocity term (computed using the second Bezier derivative) and $\beta_2 \in \mathbb{R}^2$ is the slope term.

5.3 Reduction to 2D

Substituting (51) into row 1 of (9)—the unactuated row for q_1—and solving for $\ddot{q}_1$:

$$\ddot{q}_1 = \frac{-H_1 - D_{1,2:3}\,\beta_1}{D_{11} + D_{1,2:3}\,\beta_2} \tag{52}$$

where $H_1 = C(q,\dot{q})_{1,:}\,\dot{q} + G_1$ collects the Coriolis and gravity terms. Together with $\dot{q}_1 = \dot{q}_1$, equation (52) is a self-contained 2D ODE in $z = [q_1,\, \dot{q}_1]^T$. This is the *zero dynamics*.

Notice that row 1 of the EOM has no $B_1 u$ term (since q_1 is unactuated), so the control input u does not appear in (52). The zero dynamics are autonomous—they evolve independent of what the controller does to the actuated joints.

A full walking gait requires this 2D system to have a *periodic orbit*: z must return to its starting point after one step. This is the condition we optimize.

5.4 Reconstruction of Full State from Zero Dynamics

In order to move between the two-dimensional zero dynamics embedded submanifold and the full six-dimensional state, we use the map $\Phi : \mathcal{Z} \to \mathbb{R}^6$:

$$x = \Phi(z) = \begin{bmatrix} q_1 \\ b_2(s(q_1)) \\ b_3(s(q_1)) \\ \dot{q}_1 \\ \dot{q}_1/\Delta q \cdot db_2/ds \\ \dot{q}_1/\Delta q \cdot db_3/ds \end{bmatrix} \tag{53}$$

The inverse mapping $z = \Phi^{-1}(x)$ is similarly used to go from the state space representation back to the zero dynamics manifold.

This map and inverse map are used to reconstruct the full state from the zero dynamics and vice versa, which is necessary for feedback control, visualization, and applying the impact map (which is defined in terms of the state space variables).

6 Gait Optimization

6.1 Decision Variables

The gait is parameterized by an 8-dimensional vector:

$$f = [\, z_1^-,\ z_2^-,\ \alpha_2^{q_2},\ \alpha_3^{q_2},\ \alpha_4^{q_2},\ \alpha_2^{q_3},\ \alpha_3^{q_3},\ \alpha_4^{q_3}\,] \quad (54)$$

where z_1^- and z_2^- are the pre-impact values of q_1 and $\dot{q}_1$ respectively, and the α terms are the three free Bezier coefficients for each joint (the other two are set by (42)). The symmetry $q_1^+ = -q_1^-$ is assumed, so only one pre-impact angle is needed.

6.2 Simulation Sequence for One Step

Given f, one complete step is simulated as follows:

1. Reconstruct the full pre-impact state x^- from f using the Bezier boundary values at $s = 1$.
2. Apply the impact map (35) to get x^+.
3. Extract $z^+ = [x^+(1),\, x^+(4)]^T$ (the post-impact q_1 and $\dot{q}_1$).
4. Integrate the zero dynamics (52) from z^+ until $s \geq 1$.
5. Record the final state z_f^- just before the next impact.

6.3 Objective Function

The objective for our optimization is to minimize the mechanical cost of transport (MCOT) for our robot, which can be defined as:

$$J(f) = \sum_{t_k} \dot{q}(t_k)^\top u(t_k) \quad (55)$$

where $\dot{q}$ and u only represent the actuated joints q_2, q_3. At each time sample t_k along the zero dynamics trajectory, the required torque $u(t_k)$ and $\dot{q}(t_k)$ are computed using f. Minimizing J biases the optimizer toward efficient gaits.

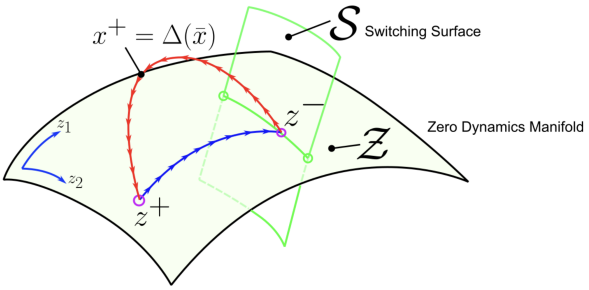

Figure 2: Hybrid Zero Dynamics (zero dynamics trajectory followed by instantaneous impact map). Figure taken from [2]

6.4 Periodicity Constraint

For the gait to be repeatable, the zero dynamics must close into a periodic orbit. This is enforced as a nonlinear equality constraint:

$$c_{eq}(f) = \begin{bmatrix} q_1^0 - z_{f,1}^- \\ \dot{q}_1^0 - z_{f,2}^- \end{bmatrix} = 0 \quad (56)$$

The pre-impact conditions at the start of the step must equal the final pre-impact conditions at the end, ensuring the gait repeats indefinitely. This concept can be seen in Figure 2.

6.5 Optimization Setup

We use MATLAB's `fmincon` with the interior-point algorithm:

- Objective: minimize (55)
- Equality constraint: periodicity (56)
- Tolerances: step 10^{-6}, function 10^{-2}
- Max iterations: 1000

The bounds for optimization were determined around a manually-tuned initial guess ($z_1^- = -0.3$ rad, $z_2^- = -1.0$ rad/s) to prevent the optimizer from wandering into physically degenerate configurations. The value for the initial orientation of the stance foot was determined by visually inspecting the resulting configuration to see if it would be a reasonable start/end to a gait cycle. We chose to initialize it with a shorter step length in order to ensure the robot would not fall backwards when the swing leg was picked up.

The initial values for $[a_2^i,\, a_3^i,\, a_4^i]$ were chosen carefully by inspecting the initial configuration we set, and determining what the start and end positions for q_2 and q_3 would be in these states. Then, setting α_0

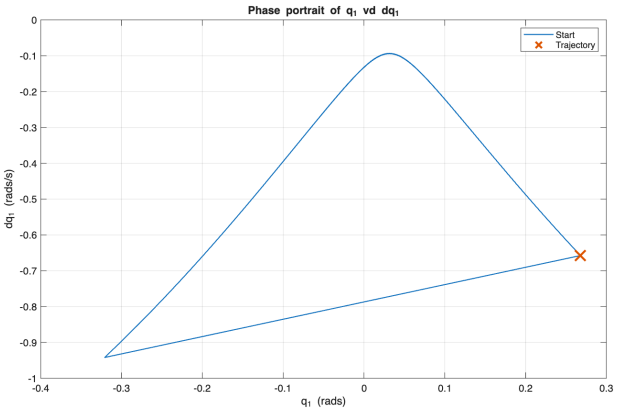

Figure 3: Phase Portrait of Zero Dynamics only, (5 gait cycles)

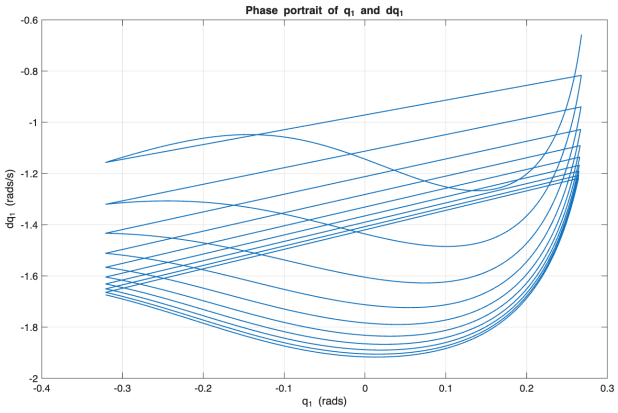

Figure 4: Phase Portrait of q_1 vs $\dot{q}_1$

and α_4 equal to these start and end values, and using equations (42), we solved for α_2. This resulted in one final free parameter, α_3, which we set by interpolating between α_2 and α_4.

This resulted in the initial conditions:

$$z^- = [-0.3, -1]$$

$$\alpha_{q_2} = [0.3, 0.4, 0.5]$$

$$\alpha_{q_3} = [\pi, \pi, \pi - z_1^-]$$

With the optimizer bounded by ± 0.1 rad for $[z_1^-,\ \alpha_{q_2},\ \alpha_{q_3}]$ and ± 1 rad/s for z_2^-

After running the gait optimization with our objective function and nonlinear constraints, and bounds, we arrived at the following parameters:

$$f = [-0.32, -0.94, 0.30, 0.46, 0.59, 3.04, 3.04, 2.83];$$

7 Full Dynamics Simulation and Validation

7.1 Zero Dynamics Validation

In order to verify that the optimizer correctly discovered a stable gait, we simulated the zero dynamics, without the impact map, for 5 gait cycles, and plotted the phase portrait of q_1 vs $\dot{q}_1$. As you can see in Figure 3, the 5 gait cycles perfectly coincide, which means that the gait is stable and periodic.

7.2 Impact Map and Hybrid Simulation

After verifying our gait design with ZD, we moved on to simulating and verifying the hybrid zero dynamics, which alternates between the continuous swing-phase dynamics and the discrete impact map. Each time $s \to 1$ (detected by the ODE event function), the states are reset by the impact map (35). These events are depicted as the straight lines in the phase portrait, marking the beginning of a new step. Additionally, this full dynamics simulation takes into account the nonlinear feedback control of u which is what actually drives the joints to their desired configuration, as described by the Bezier curves.

The resulting phase portrait from this full dynamics simulation can be seen in Figure 4.

As you can see from this full dynamics simulation, the gait does not have the same periodicity and consistency as was simulated in Figure 3. This is partly due to the fact that our robot does not have knees, and as a result, must walk downhill in order to prevent early contact with the ground. Since it is walking downhill, this figure shows a clear acceleration of the walker as it accumulates speed.

7.3 Final Results

The results of this final HZD Walking Controller can be seen in Figures 5 and 6.

The joint position and velocity trajectory plots are shown in Figure 7, showing smooth motion and relabeling of the joints as the walker moves.

The source code for this project is available on GitHub and it is accompanied by animations of our final walking controller [1].

8 Discussion

The HZD framework works well here because it cleanly separates the problem: design the orbit in 2D, then use feedback linearization to enforce it in 6D.

[1] `https://github.com/arjun-2612/LeggedRobotics_BipedalWalker`

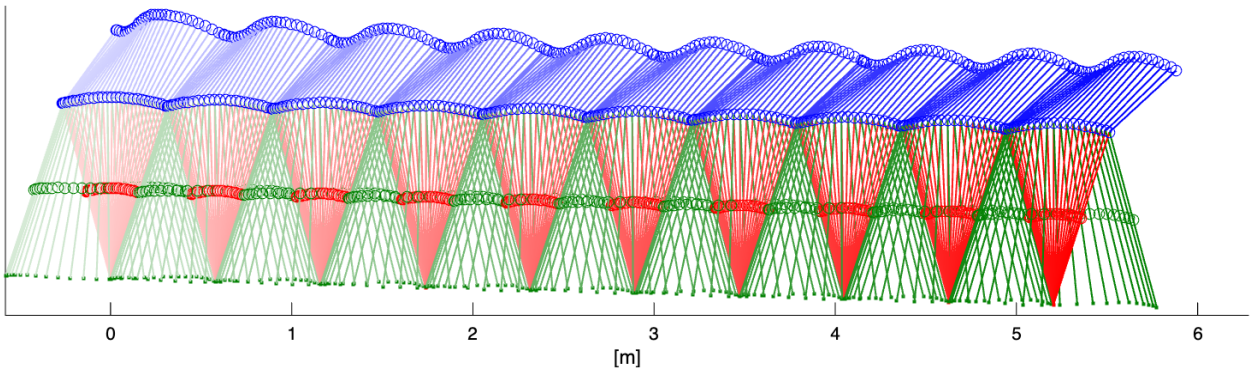

Figure 5: Walking Trajectory over 10 Steps

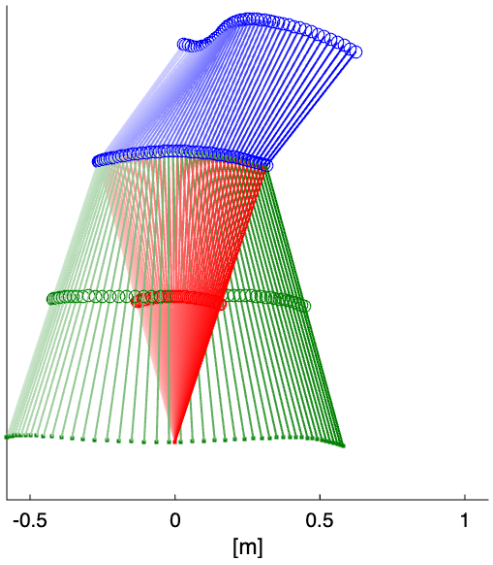

Figure 6: Trajectory over 1 Gait Cycle

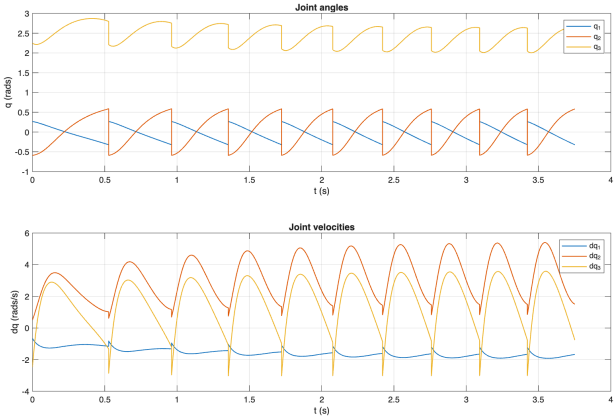

Figure 7: Position and Velocity Trajectories for all Joints in Walker over 10 steps

The Bezier parameterization is a natural fit because it gives direct control over the boundary conditions at each step transition, and the optimizer handles the rest.

8.1 Dimensionality Reduction via Virtual Constraints

The system has three degrees of freedom and two actuators, leaving q_1 unactuated. Rather than treating this as a limitation, the virtual constraint framework exploits it: q_1 serves as the independent phase variable, and the actuated joints are parameterized as functions of it. The two virtual constraints $h = 0$ and their time derivatives $L_f h = 0$ define a four-dimensional constraint on $\mathbb{R}^6$, reducing the dynamics to the two-dimensional zero dynamics manifold $\mathcal{Z}$. This reduction is exact, not approximate, and is what makes the gait optimization tractable—the optimizer searches for periodic orbits in $\mathbb{R}^2$ rather than $\mathbb{R}^6$.

8.2 Limitations

This model makes several simplifying assumptions which do not all translate well to the physical world, such as:

- Point-foot contact (no foot roll-off)
- Rigid impact (perfectly inelastic, no rebounding or slipping)
- Planar (sagittal plane only, no lateral dynamics)
- Massless feet (lumped mass at leg midpoints)
- No friction constraints (though slip would need to be checked in a more complete analysis)

With a more complex model, and added constraints for gait optimization, a lot of these issues can be fixed.

However, some of these limitations are fundamental to our model/control strategy. Shifting from a planar model to a 3-dimensional one adds a layer of complexity as the number of cyclic variables increases, and therefore optimization becomes a higher-dimensional problem that results in difficulty converging to stable parameters.

Additionally, having point feet, rather than a foot and ankle, while it allows for more dynamic movement, requires continuous movement in order to be stable. This makes these types of robots less practical in real life applications since they have to be moving constantly to stay up.

9 Conclusion

We designed and implemented a complete gait generation pipeline for a three-link planar biped using Hybrid Zero Dynamics. We derived the Lagrangian dynamics and rigid-body impact map, and reduce this problem from a 6 dimensional state space to a 2D embedded submanifold using Zero Dynamics. Then we performed optimization on this manifold, to ensure stable periodic gait cycles by solving for the Bezier coefficients and initial conditions that define it. Lastly we validated the result with the full closed-loop simulation.

The results show that this HZD approach is effective for this relatively simple model. However, there are a lot of assumptions and simplifications with this model that limit its usability and lend itself to future work. It would be interesting to expand this model to 3-dimensions, which increases the configuration space and requires lateral balance. It would also be interesting to add knee actuation to the model to prevent foot scuffing and open the door to questions about walking over different terrains, rather than exclusively downhill.

References

[1] E. R. Westervelt, J. W. Grizzle, C. Chevallereau, J. H. Choi, and B. Morris, *Feedback Control of Dynamic Bipedal Robot Locomotion*. CRC Press, 2007.

[2] A. Ramezani, "Gait design using optimization pt. 1." Course slides, Northeastern University, 2026.